ML Viva Guide: 5 Practicals Explained Simply

Nov 2, 2026 · @Vidula jadhav

Every concept, every line of code and every likely viva question for your 5 ML practicals, explained from zero with everyday examples.

Start here: what is machine learning?

Machine learning (ML) means teaching a computer by showing it many examples, instead of writing exact rules for it.

Pizza-shop example. You own a pizza shop. Old way (normal programming): you write rules like "if the pizza is large, price = 500". New way (ML): you show the computer 10,000 past orders with their prices, and it works out the pattern itself. Next time a new order comes, it predicts the price.

Three words you must know

- **Data / dataset**: the big table (like an Excel sheet). Each row is one example (one Uber ride, one email, one customer). Each column is one property.
- **Features (X)**: the columns we give the computer to look at. Example: distance, time, number of passengers.
- **Target / label (y)**: the column we want to predict. Example: the fare, or spam/not-spam.

The two big families

Family	Plain meaning	Example in your practicals
Supervised learning	Learning with the answer key. Every training row has the correct answer.	P1 (fare), P2 (spam), P3 (churn), P4 (diabetes)
Unsupervised learning	No answer key. The computer finds groups by itself.	P5 (sales clustering)

Inside supervised learning there are two kinds of prediction:

- **Regression**: predict a number (fare = 12.5 dollars). Practical 1.
- **Classification**: predict a category (spam or not, leaves or stays, diabetic or not). Practicals 2, 3, 4.

The standard recipe used in all 5 practicals

1. Load the data.
2. Clean it (missing values, wrong values, useless columns).
3. Split it into a training set (to learn from) and a test set (to be examined on).
4. Scale the numbers if needed.
5. Train a model (`.fit`).
6. Predict on the test set (`.predict`).
7. Score it (accuracy, R2, confusion matrix and so on).

School analogy. Training data is the practice questions. Test data is the final exam with questions the student has never seen. If we tested on the practice questions, a student who just memorised answers would look brilliant. That is why we always test on unseen data.

Shared toolkit: lines you will see in every practical

The libraries (the `import` lines)

Library	What it is, in child language	Where used
`pandas` (`pd`)	Excel inside Python. Holds the table as a `DataFrame` (`df`).	All 5
`numpy` (`np`)	Fast maths on numbers. Gives `np.sqrt` , `np.nan` , `np.radians` .	P1, P4, P5
`matplotlib.pyplot` (`plt`)	Draws graphs. `plt.show()` opens the picture window.	All 5
`seaborn` (`sns`)	Makes prettier graphs on top of matplotlib (boxplot, heatmap, countplot).	All 5
`sklearn` (scikit-learn)	The ML toolbox: ready-made models, splitters, scalers and score functions.	All 5
`scipy`	Science maths. We use it only for the dendrogram.	P5

Pandas lines

- `pd.read_csv('file.csv')` : reads the CSV file into a table.
- `df.shape` : (rows, columns). Example (768, 9) means 768 rows, 9 columns.
- `df.isnull().sum()` : counts empty cells in each column.
- `df.dropna()` : throws away rows that have an empty cell.
- `df.drop('col', axis=1)` : deletes a column. `axis=1` means column, `axis=0` means row.
- `df['col'].value_counts()` : counts how many times each value appears (how many spam, how many not spam).
- `df.corr()` : correlation table between all numeric columns.
- `pd.get_dummies(df, columns=[...])` : turns text categories into 0/1 columns (one-hot encoding).
- `df.groupby('Cluster').mean()` : average of every column for each cluster.

Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `test_size=0.2` : 20 percent of rows go to the exam (test), 80 percent to practice (train).
- `random_state=42` : fixes the shuffle so every run gives the same split. The number 42 has no meaning; any number works.
- `stratify=y` : keeps the same percentage of each class in both parts. Without it, all the rare cases (say, the diabetic patients) could land in one part by bad luck.

Scaling

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

- Problem: Balance is in lakhs, Age is in tens. A distance-based model would think Balance is far more important just because its numbers are bigger.
- `StandardScaler` converts every column to mean 0 and standard deviation 1. Formula: z = (x - mean) / std.
- `fit_transform` on train = learn the mean and std from training data, then apply them. `transform` on test = apply the same numbers. We never learn from test data, because that is data leakage (peeking at the exam paper).

- Needed for: KNN, SVM, neural networks, K-Means. Not needed for decision trees or random forest.

The model pattern (identical for every model)

```
model = SomeModel(settings)
model.fit(X_train, y_train)      # learn
p = model.predict(X_test)        # guess answers for unseen rows
score_function(y_test, p)        # compare guesses with the real answers
```

Metrics for classification (the confusion matrix)

A classifier makes 4 kinds of results. Example: spam detector, where spam is the positive class.

	Predicted Not Spam	Predicted Spam
Actually Not Spam	TN (true negative): correct	FP (false positive): a good mail wrongly sent to spam
Actually Spam	FN (false negative): a spam mail that slipped through	TP (true positive): spam caught

```
\text{Accuracy}=\frac{TP+TN}{TP+TN+FP+FN}\qquad \text{Precision}=\frac{TP}{TP+FP}\qquad \text{Recall}=\frac{TP}{TP+FN}\qquad F1=\frac{2\cdot P\cdot R}{P+R}
```

- **Accuracy**: out of all predictions, how many were right.
- **Error rate**: 1 minus accuracy.
- **Precision**: when I say "positive", how often am I right? (A careful friend who rarely cries wolf.)
- **Recall (sensitivity)**: out of all real positives, how many did I find? (A strict security guard who catches everyone.)
- **F1**: one number balancing precision and recall.
- **Why not just accuracy?** If 95 of 100 emails are normal, a lazy model that always says "normal" gets 95 percent accuracy yet catches zero spam. Always check precision and recall on imbalanced data.

In sklearn the confusion matrix prints as `[[TN, FP], [FN, TP]]`.

Metrics for regression (Practical 1)

```
MAE=\frac{1}{n}\sum |y-\hat y|\qquad MSE=\frac{1}{n}\sum (y-\hat y)^2\qquad
RMSE=\sqrt{MSE}\qquad R^2=1-\frac{\sum (y-\hat y)^2}{\sum (y-\bar y)^2}
```

- **MAE**: average size of the mistake, in fare units. Easy to explain.
- **RMSE**: like MAE but punishes big mistakes more, still in fare units.
- **R2**: what share of the ups and downs in fare the model explains. 1 is perfect, 0 is as bad as always guessing the average, negative is worse than that.

Practical 1: Uber fare prediction (regression)

Goal in one line: given pickup and drop-off places and the time, predict the fare (a number). So this is supervised **regression**.

The concepts, one by one

1. Pre-processing (cleaning the data). Real data is dirty, like a messy classroom register. We fix:

- *Useless columns*: `Unnamed: 0` and `key` are just ID numbers. They say nothing about the fare, so we drop them.
- *Missing values*: a few rows have no drop-off location. We delete those rows with `dropna()`.
- *Wrong data type*: `pickup_datetime` is stored as plain text. `pd.to_datetime` turns it into a real date so we can pull out hour, day, month and year. Reason: fares are higher at rush hour, so the hour matters.
- *Impossible values*: fare of 0 or negative, 0 passengers, latitude outside -90 to 90. We filter these out.

2. Feature engineering (making a smarter column). Fare depends mainly on how far you travel, but the data only has latitude and longitude. We compute **distance** from them using the **Haversine formula**, which gives the straight-line distance between two points on a sphere (the Earth). Radius of Earth = 6371 km.

3. Outliers. An outlier is a value far away from everyone else, like one student scoring 100 when the class average is 30, or a fare of 500 dollars for 1 km. They drag a model off course, especially linear regression.

- *Boxplot*: a picture of the data. The box is the middle 50 percent. Dots outside the whiskers are outliers.

- *IQR method*: Q1 = 25th percentile, Q3 = 75th percentile, IQR = Q3 - Q1. Anything below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR is an outlier. Example: Q1 = 6, Q3 = 12, IQR = 6, so the allowed range is -3 to 21. A fare of 40 is removed.
- Other methods: Z-score (more than 3 standard deviations from the mean), and capping/winsorising (replace extreme values with the limit instead of deleting).

4. Correlation. A number from -1 to +1 saying how strongly two columns move together.

- +1: when one goes up, the other goes up (distance and fare).
- 0: no relationship.
- -1: when one goes up, the other goes down.
- A **heatmap** colours this table so you can spot strong pairs at a glance. We want features that correlate with `fare_amount`. Distance is the strongest.
- Correlation is not causation, and Pearson correlation only sees straight-line relationships.
- *Multicollinearity*: two features that are strongly correlated with each other (for example pickup and dropoff longitude). It can make linear regression unstable.

5. Linear regression. Draws the best straight line through the data: fare = b0 + b1 x distance + b2 x hour + ... It picks the b values that make the total squared error smallest (**least squares**). Simple, fast, easy to explain, but it assumes straight-line relationships.

6. Random forest regression. A forest is many decision trees. Each tree is trained on a random sample of the rows and a random subset of the columns, and gives its own price guess. The forest **averages** all the guesses. This is called *bagging* (bootstrap aggregating) and it is an *ensemble* method.

- *Decision tree*: a flowchart of yes/no questions. "Is distance more than 5 km? Yes. Is it rush hour? Yes. Fare = 18."
- One tree can overfit. Averaging many different trees cancels their individual mistakes (the wisdom of the crowd).
- It captures curves and interactions, and is not hurt much by outliers, so it usually beats linear regression here.

The code, line by line

Code	Meaning
`df.drop(['Unnamed: 0','key'], axis=1)`	remove the two ID columns

Code	Meaning
`df.isnull().sum()` then `df.dropna()`	count empty cells, then delete those rows
`pd.to_datetime(...)`	text to real date
`.dt.hour`, `.dt.dayofweek`, `.dt.month`, `.dt.year`	pull parts out of the date (dayofweek: Monday = 0)
`haversine(...)`	our own function that returns km; `np.radians` converts degrees to radians because sin and cos need radians
`df[(df.fare_amount>0) & (df.distance>0) & ...]`	keep only the sensible rows (`&` means AND)
`sns.boxplot(...)` before and after	shows the outliers, then shows them gone
`remove_outliers(d, col)`	IQR rule; `quantile([.25,.75])` gives Q1 and Q3; `between(low, high)` keeps rows inside
`sns.heatmap(df.corr(), annot=True)`	coloured correlation table with the numbers printed
`X = df.drop('fare_amount', axis=1)`, `y = df['fare_amount']`	features and target
`train_test_split(..., test_size=0.2)`	80 percent train, 20 percent test
`LinearRegression()`	the straight-line model
`RandomForestRegressor(n_estimators=50, n_jobs=-1)`	50 trees; `n_jobs=-1` uses all CPU cores
`r2_score`, `mean_squared_error`, `mean_absolute_error`	the three scores; `np.sqrt(MSE)` gives RMSE

Why no scaling here? Random forest does not need it, and plain linear regression still gives the same predictions with or without scaling.

Expected result and how to talk about it

Random forest should show a higher R2 (about 0.8 versus 0.7) and lower RMSE and MAE than linear regression. Say: "Random forest performed better because the fare does not depend on distance in a perfectly straight line, and the forest learns non-linear patterns."

Viva questions with answers

1. **Why regression, not classification?** The fare is a continuous number.
2. **What does `random_state` do?** It fixes the random shuffle so results repeat.
3. **Why did you make the distance column?** It is the strongest driver of fare and the raw coordinates do not show it directly.
4. **Why remove outliers?** They pull the fitted line towards them and inflate error.
5. **IQR formula?** IQR = Q3 - Q1; limits = Q1 - 1.5 IQR and Q3 + 1.5 IQR.
6. **What does R2 = 0.8 mean?** The model explains 80 percent of the variation in fare.
7. **Why is RMSE higher than MAE?** RMSE squares errors first, so big errors count more.
8. **Can R2 be negative?** Yes, when the model is worse than just predicting the mean.
9. **What are `n_estimators`?** The number of trees in the forest.
10. **Does random forest overfit?** Much less than a single tree, because averaging cancels noise, but a very deep forest on small data still can.
11. **How would you improve the models?** Hyperparameter tuning (`GridSearchCV`), cross-validation, more features (pickup in a busy zone, airport flag), gradient boosting (XGBoost).
12. **Which assumptions does linear regression make?** Linearity, independent errors, constant error variance (homoscedasticity), roughly normal errors, and little multicollinearity.
13. **Why does the data need to be split before judging the model?** To measure performance on unseen data and detect overfitting.

Practical 2: Email spam detection (KNN and SVM)

Goal in one line: look at an email and say Spam (1, the abnormal state) or Not Spam (0, the normal state). Two possible answers means **binary classification**.

The concepts, one by one

1. The dataset. `emails.csv` has about 5172 emails. Each email is one row. The 3000 columns are the counts of the 3000 most common words ("free" appears 4 times, "meeting" 0 times, ...). The last column `Prediction` is the answer. This idea of turning text into word counts is called **bag of words**: we ignore grammar and order and only count words, like tipping a bag of Scrabble tiles on the table.

2. Imbalanced data. About 71 percent of emails are Not Spam and 29 percent are Spam. Accuracy alone can fool us here, so we also report precision, recall and F1.

3. KNN (K-Nearest Neighbours). The simplest idea in ML: *tell me who your neighbours are and I will tell you who you are.*

- Put every training email as a point in space (3000 dimensions here).
- For a new email, find the **K closest** training emails (K = 5).
- Take a vote. If 4 of the 5 neighbours are spam, call it spam.
- *Distance* = Euclidean distance, the straight-line ruler distance: sqrt of the sum of squared differences.
- *Choosing K*: too small (K=1) copies noise and overfits; too large blurs everything and underfits. Use an odd K to avoid tie votes. Try several K and keep the best.
- *Lazy learner*: there is no training. It simply stores the data and does all the work when asked to predict. That makes prediction slow.
- *Weakness here*: with 3000 columns, almost every point looks equally far from every other. This is the **curse of dimensionality**.

Real-life example: you move to a new colony and want to know if it is safe. You ask your 5 nearest neighbours. Majority says safe, so you conclude safe.

4. SVM (Support Vector Machine). Draw the best possible dividing line between the two groups.

- Many lines can separate spam from not-spam. SVM picks the one with the **widest gap (margin)** between the classes, like the widest road between two villages.
- The few points sitting on the edge of the road are the **support vectors**. Only they decide the line; the other points could be deleted without changing it.
- In more than 2 dimensions the line becomes a **hyperplane**.
- **Kernel**: a trick that lets SVM handle data that cannot be split by a straight line by lifting it into a higher dimension. Types: `linear`, `poly`, `rbf` (the common default for curved boundaries). For text with thousands of features, `linear` works best and is fastest.

- **C parameter**: how strict SVM is about mistakes. Large C = few mistakes allowed, narrow margin, risk of overfitting. Small C = wider margin, a few mistakes tolerated.

5. Evaluating. See the confusion matrix and formulas in the Shared toolkit section. For spam:

- A **false positive** (real mail sent to spam) is annoying and can be dangerous (a job offer lost). So we care about precision.
- A **false negative** (spam in the inbox) is less harmful. Recall measures how much spam we catch.

The code, line by line

Code	Meaning
`pd.read_csv('emails.csv')`	load the table
`df['Prediction'].value_counts()`	how many spam and not spam
`df.dropna().drop('Email No.', axis=1)`	remove empty rows and the ID column
`X = df.drop('Prediction', axis=1)`, `y = df['Prediction']`	3000 word-count features and the label
`sns.countplot(x=y)`	bar chart of the two classes
`train_test_split(X, y, test_size=0.2, random_state=42)`	80/20 split
`KNeighborsClassifier(n_neighbors=5)`	KNN with K = 5
`SVC(kernel='linear')`	SVM with a straight-line boundary
`models = {...}` and `for name, m in models.items():`	store both models in a dictionary and run the same code for each
`m.fit(X_train, y_train)`	learn
`p = m.predict(X_test)`	predict the test emails
`accuracy_score`, `precision_score`, `recall_score`, `f1_score`	the four scores
`classification_report`	precision, recall and F1 for each class together

Code	Meaning
`confusion_matrix` + `sns.heatmap(..., annot=True, fmt='d')`	the 2x2 table drawn as a coloured picture

We did not scale here because every column is already a word count in a similar range.

Expected result and how to talk about it

SVM about 95 to 98 percent accuracy, KNN about 85 to 90 percent. Say: "SVM performed better because it works well on high-dimensional sparse text data, whereas KNN suffers from the curse of dimensionality and is also slower at prediction time."

Viva questions with answers

1. **What is binary classification?** Two output classes. Here Normal (0) and Abnormal/Spam (1).
2. **KNN versus SVM, in one line each?** KNN votes among the K nearest points; SVM finds the maximum-margin boundary.
3. **Why is KNN a lazy learner?** No model is built during `fit`; it only stores data.
4. **What distance does KNN use?** Euclidean by default; Manhattan and Minkowski are alternatives.
5. **What are support vectors?** The points closest to the boundary that define the margin.
6. **What is a kernel?** A function that implicitly maps data to a higher dimension so a linear boundary can separate it.
7. **Why is accuracy misleading here?** The classes are imbalanced (71/29).
8. **Which is worse, FP or FN, in spam filtering?** FP: a genuine email lost. Hence high precision matters.
9. **What is the F1 score?** The harmonic mean of precision and recall.
10. **Does KNN need feature scaling?** Yes in general, because it relies on distances. Here the features are all counts, so the effect is small.
11. **How do you pick K?** Try K = 1 to 20 and plot accuracy against K, or use cross-validation.
12. **Can SVM do multi-class?** Yes, using one-vs-one or one-vs-rest.
13. **What is bag of words?** Representing text by word counts, ignoring order. A better version is TF-IDF, which down-weights very common words.

14. **How could you improve the spam model?** Tune C and kernel with `GridSearchCV`, use TF-IDF, try Naive Bayes (the classic spam algorithm), handle imbalance with class weights.

Practical 3: Bank customer churn (neural network)

Goal in one line: predict whether a bank customer will leave (`Exited` = 1) or stay (0). "Churn" means customers leaving. Binary classification using a **neural network**.

The concepts, one by one

1. What is a neural network? A model loosely inspired by the brain. It is built from layers of small calculators called **neurons**.

- **Input layer**: one node per feature (CreditScore, Age, Balance, ...). Here 11 inputs after encoding.
- **Hidden layers**: the thinking part. Each neuron looks at all the previous layer's outputs and passes on a result.
- **Output layer**: one neuron giving the probability of leaving (above 0.5 means leaves).

2. What one neuron does. Think of a student deciding whether to go to a party. Weather, friends going and exam tomorrow each get an *importance* (weight). He adds them up, adds his personal mood (bias), then decides yes or no.

```
\text{output}=f\big(w_1x_1+w_2x_2+\dots+w_nx_n+b\big)
```

- `w` = weights (learned importance), `b` = bias, `f` = **activation function**.

3. Activation functions. Without them the whole network collapses into one straight line. They add curves (non-linearity).

Name	Formula	Used for
ReLU	max(0, x)	hidden layers; fast, avoids vanishing gradient
Sigmoid	1 / (1 + e^-x), squashes to 0 to 1	output layer in binary classification
Tanh	squashes to -1 to 1	older hidden layers
Softmax	probabilities that add up to 1	output layer for many classes

4. How a network learns (training).

1. **Forward pass**: data flows through layers and produces a prediction.
2. **Loss**: measure how wrong it was. Binary classification uses **binary cross-entropy (log loss)**.
3. **Backpropagation**: work backwards from the output, finding how much each weight contributed to the error (using gradients and the chain rule).
4. **Gradient descent**: nudge each weight a small step in the direction that reduces the loss. Think of walking downhill in fog. The step size is the **learning rate**.
5. Repeat. One full pass over all training rows = one **epoch**. Rows are processed in small groups called **batches**.

Optimizer: the algorithm that does the stepping. **Adam** is the popular one, with an adaptive learning rate per weight. Others: SGD, RMSprop.

5. Pre-processing for this dataset.

- Drop `RowNumber`, `CustomerId`, `Surname`: identifiers, no predictive value.
- **One-hot encoding** with `get_dummies`: the network only understands numbers. Geography (France, Spain, Germany) becomes two 0/1 columns `Geography_Germany` and `Geography_Spain` (France = both 0). `drop_first=True` removes the redundant column, avoiding the *dummy variable trap* (one column perfectly predictable from the others).
- **Normalisation** with `StandardScaler`. Essential for neural networks: unscaled inputs make training slow and unstable because Balance (about 100,000) would swamp Age (about 40). Fit on train only, then transform test (avoids leakage).
- `stratify=y` keeps the 80/20 stay/leave ratio in both splits.

6. Improving the model (step 4 of the question).

Change	Why it helps
More neurons and layers, `(6,)` to `(32, 16)`	can learn more complex patterns
ReLU activation	faster, avoids vanishing gradients
Adam optimizer	adaptive and stable
`early_stopping=True`	stops when validation score stops improving, preventing overfitting
`alpha=0.001` (L2 regularisation)	penalises very large weights, keeps the model simple

Change	Why it helps
Not in code but mention: dropout, class weights or SMOTE for imbalance, tuning with `GridSearchCV`, more epochs, batch normalisation	

7. Overfitting and underfitting.

- *Overfitting*: memorises training data, fails on new data (like a student who memorises last year's paper). Training score high, test score low.
- *Underfitting*: too simple to learn anything (training score also low).
- Fixes for overfitting: more data, simpler model, regularisation, dropout, early stopping.

The code, line by line

Code	Meaning
`pd.read_csv('Churn_Modelling.csv')`	load the 10,000 customers
`df['Exited'].value_counts()`	about 7963 stay, 2037 leave
`df.drop(['RowNumber','CustomerId','Surname'], axis=1)`	remove ID columns
`pd.get_dummies(df, columns=['Geography','Gender'], drop_first=True)`	text to 0/1
`X = df.drop('Exited', axis=1)`, `y = df['Exited']`	features and target
`train_test_split(..., stratify=y)`	80/20 split with equal class mix
`StandardScaler().fit_transform` / `.transform`	normalise
`MLPClassifier(hidden_layer_sizes=(6,), max_iter=300)`	MLP = Multi-Layer Perceptron; one hidden layer of 6 neurons; `max_iter` = maximum epochs
`MLPClassifier(hidden_layer_sizes=(32,16), activation='relu', solver='adam', alpha=0.001, early_stopping=True)`	the improved network

Code	Meaning
`accuracy_score`, `confusion_matrix`, `classification_report`	scores (the 5 points asked in the question)
`model.loss_curve_`	loss after each iteration; the line should fall

If your teacher expects Keras: the same network is `Sequential([Dense(32, activation='relu'), Dense(16, activation='relu'), Dense(1, activation='sigmoid')])`, compiled with `optimizer='adam', loss='binary_crossentropy'` and trained with `model.fit(X_train, y_train, epochs=100, batch_size=32)`. Say this in the viva to show you know both.

Expected result and how to talk about it

Accuracy about 85 to 87 percent. Confusion matrix example `[[1540, 53], [226, 181]]`: 1540 stayers predicted right, 53 stayers wrongly flagged, 226 leavers missed, 181 leavers caught. Say: "Accuracy is good but recall for the leaving class is low because the data is imbalanced; I would improve it with class weights or SMOTE."

Viva questions with answers

1. **What is a perceptron?** The simplest neuron: weighted sum plus bias through a step activation. A multi-layer perceptron stacks many.
2. **What is a hidden layer?** Any layer between input and output.
3. **Why activation functions?** To add non-linearity so the network can learn complex patterns.
4. **What is backpropagation?** Sending the error backwards to compute gradients and update weights.
5. **What is gradient descent?** Repeatedly moving weights in the direction that lowers the loss.
6. **What is the learning rate?** The step size of each update. Too big overshoots; too small is slow.
7. **Epoch versus batch?** Epoch = one pass over all data; batch = a small group of rows used for one update.
8. **What is the vanishing gradient problem?** In deep sigmoid/tanh networks, gradients shrink to near zero so early layers stop learning. ReLU helps.

9. **Why scale inputs for a neural network?** Faster, stabler convergence and no feature dominates.
10. **Why is accuracy not enough here?** 80 percent of customers stay; predicting "stay" for everyone already gives 80 percent.
11. **Which error costs more, FN or FP?** FN: a customer we failed to flag leaves and the bank loses revenue.
12. **What is dropout?** Randomly switching off some neurons during training so the network cannot rely on any one of them, which reduces overfitting.
13. **What is the dummy variable trap?** Redundant one-hot columns that are perfectly correlated; removed using `drop_first=True`.
14. **How many neurons in the output layer, and why?** One, with sigmoid, giving the probability of leaving.
15. **Why did you use `random_state=42`?** To get repeatable results.

Practical 4: Diabetes prediction with KNN

Goal in one line: given a patient's medical readings, predict whether they have diabetes (`Outcome` = 1) or not (0), using KNN, then compute the confusion matrix, accuracy, error rate, precision and recall.

The concepts, one by one

1. The dataset (Pima Indians). 768 patients, 8 features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction (family history score), Age. About 500 non-diabetic and 268 diabetic, so mildly imbalanced.

2. Hidden missing values. Look at Glucose = 0 or BMI = 0. A living person cannot have zero glucose, so a 0 here means "not recorded". `isnull()` will not find it, because the cell is not empty. We replace those zeros with `NaN` and then fill with the **median**.

- Median = the middle value when sorted. Preferred over the mean because a few extreme values do not move it. Example: salaries 10, 12, 14, 16, 500. Mean = 110 (misleading), median = 14.
- Other ways to handle missing data: drop rows, fill with mean or mode, KNN imputer, or predict the value with a model.

3. KNN recap. For a new patient, find the K = 5 most similar patients (closest by Euclidean distance over the 8 readings) and take a majority vote on diabetic or not.

- **Scaling is compulsory**: Insulin ranges up to about 800 while Pregnancies is 0 to 17. Without scaling, Insulin alone decides the distance.
- *Why odd K?* A 5-neighbour vote can never tie 2.5 to 2.5.
- *Bias-variance view*: small K = low bias, high variance (jumpy). Large K = high bias, low variance (smooth).
- The bonus graph "Accuracy vs K" lets you pick the best K by eye.

4. The four numbers you must compute. Example with real-looking numbers: the confusion matrix is `[[85, 15], [27, 27]]` for 154 test patients.

	Predicted No	Predicted Yes
Actual No	TN = 85	FP = 15
Actual Yes	FN = 27	TP = 27

- Accuracy = (27 + 85) / 154 = 0.727
- Error rate = 1 - 0.727 = 0.273 (or (FP + FN) / total = 42 / 154)
- Precision = TP / (TP + FP) = 27 / 42 = 0.64
- Recall = TP / (TP + FN) = 27 / 54 = 0.50

Reading it in plain English: of the patients we called diabetic, 64 percent really were. Of all the actually diabetic patients, we caught only half. In medicine, **recall matters more**, because missing a sick patient (false negative) is worse than a false alarm. Your own run printed precision 0.66 and recall 0.6111, which is in the same range, and you can explain it exactly like this.

The code, line by line

Code	Meaning
`cols = [...]`	the five columns where 0 is impossible
`df[cols].replace(0, np.nan)`	mark zeros as missing
`.fillna(df[cols].median())`	fill each column's gaps with its own median
`train_test_split(..., stratify=y)`	split keeping diabetic share equal
`StandardScaler`	scale before KNN

Code	Meaning
`KNeighborsClassifier(n_neighbors=5)`	the model
`knn.fit` and `knn.predict`	learn, then predict
`confusion_matrix(y_test, p)`	the 2x2 table
`1 - acc`	error rate
`precision_score`, `recall_score`	the two remaining metrics
list comprehension over `range(1, 21)` with `.score`	tries K = 1 to 20 and returns the test accuracy of each

Viva questions with answers

1. **Is KNN parametric?** No. It makes no assumption about the data shape; it is non-parametric and instance-based.
2. **What is the time complexity concern?** Prediction compares the new point to every training point, so it is slow on big data. Tree structures (KD-tree, ball tree) speed it up.
3. **What is the difference between accuracy and precision?** Accuracy covers all predictions; precision only looks at those we predicted positive.
4. **Define sensitivity and specificity.** Sensitivity = recall = TP/(TP+FN). Specificity = TN/(TN+FP): how many healthy people we correctly cleared.
5. **Why not use mean to fill the zeros?** Skewed columns like Insulin have outliers; the median is robust.
6. **What does `stratify` do?** Keeps class ratio the same in train and test.
7. **Why scale only after splitting?** To avoid data leakage.
8. **What if K = 1?** Perfect training accuracy but noisy, overfitted predictions.
9. **Other distance measures?** Manhattan (sum of absolute differences), Minkowski (general form), cosine, Hamming for categories.
10. **What is the ROC curve and AUC?** ROC plots true positive rate against false positive rate at every threshold; AUC is the area under it, 1 is perfect and 0.5 is random guessing.
11. **How can recall be improved?** Lower the decision threshold, use class weights, try a different model, or collect more diabetic examples.
12. **What is cross-validation?** Splitting data into several folds and rotating which fold is the test, giving a more reliable score than one split.

Practical 5: Sales clustering (K-Means, elbow, hierarchical)

Goal in one line: group similar sales orders together without any labels, and decide how many groups there should be using the elbow method. This is **unsupervised learning**, so there is no `y` and no train/test split.

The concepts, one by one

1. Clustering. Sorting things into groups so that items in a group are alike and items in different groups are different. Example: a teacher sorts 100 unlabeled photos into piles of beaches, mountains and cities, without anyone naming the piles. Real uses: customer segmentation, grouping news, compressing images.

2. K-Means, step by step. K is the number of groups you want.

1. Choose K random points as the starting **centroids** (group centres).
2. Give every data point to its nearest centroid. Now you have K groups.
3. Move each centroid to the **mean** (average position) of its group.
4. Repeat steps 2 and 3 until the centroids stop moving.

Example with students: K = 3 on marks and attendance gives "high marks high attendance", "low marks low attendance" and "in between" groups.

- `n_init=10`: run the whole algorithm 10 times from different random starts and keep the best, because a bad start gives a bad result.
- `random_state=42`: repeatable results.
- Weaknesses: you must choose K, it is sensitive to outliers and scale, prefers round equal-size clusters, and different starts can give different answers.
- Improved start: **K-Means++** (the sklearn default) spreads the initial centroids apart.

3. Scaling is a must. K-Means measures distances. SALES (hundreds to thousands) would overpower QUANTITYORDERED (tens). So we use `StandardScaler`.

4. The elbow method (the main topic of this practical). How do we pick K?

- **WCSS** (within-cluster sum of squares, called `inertia_` in sklearn) = the sum of squared distances from every point to its own centroid. Small WCSS means tight groups.
- Run K-Means for K = 1, 2, ... 10 and write down WCSS each time.

- Plot K on the x-axis and WCSS on the y-axis. The curve falls steeply and then flattens. The bend, like the elbow of your arm, is the best K: after it, extra clusters buy very little improvement.
- Why not choose the lowest WCSS? It always keeps falling as K grows. At K = number of points, WCSS is 0 and the answer is useless.
- In your data the elbow is usually at K = 3 or 4.

5. Silhouette score (a second opinion). For each point it compares how close it is to its own cluster against the nearest other cluster. Range -1 to +1. Near 1 = well separated; near 0 = overlapping; negative = probably in the wrong cluster. Another method to pick K: choose the K with the highest average silhouette.

6. Hierarchical clustering (the second option in the question). Builds a family tree of clusters, so you do not need to choose K first.

- **Agglomerative (bottom-up)**: start with every point alone, repeatedly merge the two closest clusters, until only one remains. This is what `AgglomerativeClustering` does.
- **Divisive (top-down)**: start with one big cluster and keep splitting. Rarely used.
- **Dendrogram**: the tree picture. The height of a join = the distance at which the two clusters merged. **Cut the tree with a horizontal line** at a height where the vertical lines are longest; the number of lines the cut crosses is the number of clusters.
- **Linkage** = how we measure distance between two clusters: *single* (closest pair), *complete* (farthest pair), *average*, and *ward* (merge the pair that increases total variance least; the most common and gives compact clusters).
- Costly on big data (about n squared memory), which is why we plot the dendrogram on only 200 rows.

	K-Means	Hierarchical
Need K beforehand?	Yes	No (cut the dendrogram)
Speed on big data	Fast	Slow
Result	Flat groups	Tree of groups
Randomness	Depends on start	Same result every time
Cluster shape	Round	Depends on linkage

The code, line by line

Code	Meaning
`pd.read_csv(..., encoding='latin1')`	the file has special characters that UTF-8 cannot read; latin1 can
`df[['QUANTITYORDERED','PRICEEACH','SALES','MSRP']]`	choose 4 numeric columns; clustering needs numbers, not text or dates
`X.dropna()`	drop empty rows
`StandardScaler().fit_transform(X)`	scale (no split, so we use all the data)
`for k in range(1, 11): ... KMeans(n_clusters=k).fit(X_scaled).inertia_`	the elbow loop, collecting WCSS values in a list
`plt.plot(range(1,11), wcss, marker='o')`	the elbow graph
`k = 3`	the K you read off the graph
`km.fit_predict(X_scaled)`	fit and return the cluster number (0, 1, 2) for each row
`X['Cluster'] = labels`	attach the cluster number to each row
`X.groupby('Cluster').mean()`	average profile of each cluster, which lets you name them: small orders, medium orders, big orders
`silhouette_score(X_scaled, labels)`	quality score
`plt.scatter(..., c=labels)`	colour points by cluster
`linkage(sample, method='ward')` and `dendrogram(...)`	build and draw the tree (scipy)
`AgglomerativeClustering(n_clusters=k, linkage='ward')`	hierarchical clustering with K clusters

Expected result and how to talk about it

Elbow at 3 or 4, silhouette about 0.3 to 0.45. Say: "I used the elbow method to choose K = 3. The clusters represent low-value, medium-value and high-value orders, which the business could target differently."

Viva questions with answers

1. **Why is there no accuracy score?** No true labels exist, so we judge with inertia and silhouette.
2. **What is a centroid?** The mean point of a cluster.
3. **What is inertia?** Same as WCSS.
4. **Why does WCSS always fall as K rises?** More centres mean every point sits closer to one.
5. **What if the elbow is not clear?** Use the silhouette score, or the gap statistic, or business knowledge.
6. **Is K-Means guaranteed to find the best answer?** No, only a local optimum; hence `n_init`.
7. **How do outliers affect K-Means?** They pull centroids; remove them or use K-Medoids or DBSCAN.
8. **What is DBSCAN?** A density-based method that finds clusters of any shape, labels isolated points as noise, and needs no K.
9. **Difference between clustering and classification?** Classification has labels and learns to predict them; clustering discovers groups with no labels.
10. **How do you evaluate clusters without labels?** Silhouette score, Davies-Bouldin index (lower is better), Calinski-Harabasz index (higher is better).
11. **What does agglomerative mean?** Merging bottom-up.
12. **Distance used by K-Means?** Euclidean.

Extra ML concepts a final-year engineer should know

Examiners love to step outside the practical. These are the most likely topics, each with a child-level explanation.

Core ideas

Concept	Plain explanation
Overfitting	Memorising the practice paper. High train score, low test score.
Underfitting	Too lazy to learn. Low score on both.

Concept	Plain explanation
Bias-variance trade-off	Bias = too simple, always misses (underfit). Variance = too sensitive to the training data (overfit). We want the sweet spot.
Cross-validation (k-fold)	Split data into k parts; train on k-1, test on the 1 left; rotate k times; average. A fairer score than one split.
Hyperparameter	A setting you choose before training (K in KNN, C in SVM, number of trees). Different from parameters (weights) the model learns.
Grid search / random search	Try many hyperparameter combinations automatically and pick the best (`GridSearchCV`).
Data leakage	Information from the test set sneaking into training (for example scaling before splitting). Gives fake high scores.
Feature engineering	Creating better columns, like distance from coordinates.
Feature selection	Dropping columns that do not help.
Curse of dimensionality	With very many features, data becomes sparse and distances stop meaning much.

Regularisation (stopping overfitting)

- **L1 (Lasso)**: penalty on the sum of absolute weights; can shrink some weights exactly to zero, so it also selects features.
- **L2 (Ridge)**: penalty on the sum of squared weights; shrinks all weights smoothly. `alpha` in `MLPClassifier` is this.
- **Dropout, early stopping**: for neural networks.

Handling imbalanced data: class weights, oversampling the small class (SMOTE creates synthetic examples), undersampling the large class, and judging by precision, recall, F1 and AUC instead of accuracy.

Encoding and scaling

- **Label encoding**: categories to 0, 1, 2 (only for ordered categories such as Low, Medium, High).

- **One-hot encoding**: one 0/1 column per category (for unordered ones such as country).
- **Standardisation**: mean 0, std 1. **Min-Max scaling**: squeeze into 0 to 1 using (x - min)/(max - min). Use Min-Max when you want a fixed range; standardisation when outliers exist or data is roughly bell-shaped.

Other algorithms worth one line each

Algorithm	One-line idea
Logistic regression	Despite the name, a classifier: a straight-line score pushed through a sigmoid to give a probability.
Decision tree	A flowchart of yes/no questions; splits chosen by Gini impurity or entropy (information gain).
Random forest	Many trees on random data, vote or average (bagging).
Boosting (AdaBoost, Gradient Boosting, XGBoost)	Build trees one after another, each fixing the previous one's mistakes. Often the winner on tables of data.
Naive Bayes	Uses Bayes' theorem and assumes features are independent; fast, a classic for spam.
PCA	Compresses many columns into a few new ones that keep most of the information; used for visualisation and speed-up.
DBSCAN	Clusters by density, finds odd shapes and noise.

Bagging versus boosting: bagging trains many models in parallel on random samples and averages them (reduces variance). Boosting trains models in sequence, each focusing on earlier errors (reduces bias).

Extra evaluation ideas

- **ROC curve and AUC**: summarise a classifier across all thresholds. AUC 1 is perfect, 0.5 is a coin toss.

- **Precision-recall trade-off**: raising the threshold gives higher precision and lower recall, and the opposite when lowering it.
- **Adjusted R2**: R2 corrected for the number of features, so adding useless columns does not look like progress.
- **Train / validation / test**: validation set is for tuning, test set is touched once at the very end.

Deep learning words

- **CNN**: neural network for images. **RNN / LSTM**: for sequences such as text and time series. **Transformer**: the architecture behind modern language models.
- **Epoch, batch size, learning rate, optimizer, loss function**: the five training knobs.
- **Vanishing and exploding gradients**: gradients too small or too big when passing through many layers.

Machine learning versus deep learning versus AI: AI is the big idea of smart machines; ML is learning from data inside AI; deep learning is ML using many-layered neural networks.

Types of learning: supervised, unsupervised, semi-supervised, and reinforcement learning (an agent learns by trial, reward and penalty, like training a dog with treats).

The ML project life cycle: define the problem, collect data, clean data, explore (EDA), build features, split, train, evaluate, tune, deploy, monitor.

Last-night cheat sheet and exam-room tips

All five practicals in one table

#	Task	Type	Models	Key preprocessing	Key metrics	Remember
1	Uber fare	Supervised regression	Linear regression, random forest	Drop IDs, dropna, datetime parts, Haversine distance, IQR outliers	R2, RMSE, MAE	Random forest wins
2	Spam email	Supervised binary classification	KNN (K=5), SVM (linear)	Drop Email No.	Accuracy, precision, recall, F1, confusion matrix	SVM wins on text

#	Task	Type	Models	Key preprocessing	Key metrics	Remember
3	Bank churn	Supervised binary classification	Neural network (MLP)	Drop IDs, one-hot encode, StandardScaler	Accuracy, confusion matrix	Basic then improved network
4	Diabetes	Supervised binary classification	KNN	Replace zeros with median, StandardScaler	Confusion matrix, accuracy, error rate, precision, recall	Recall matters most
5	Sales clusters	Unsupervised clustering	K-Means, hierarchical	Pick numeric columns, StandardScaler	WCSS (elbow), silhouette	Elbow at K = 3 or 4

Five lines that appear everywhere

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model.fit(X_train, y_train)
p = model.predict(X_test)
accuracy_score(y_test, p)        # classification
r2_score(y_test, p)              # regression
```

Formulas to say without thinking

- Accuracy = (TP + TN) / total. Error rate = 1 - accuracy.
- Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1 = 2PR / (P + R).
- RMSE = square root of the mean squared error. R2 = 1 - (error of model / error of always guessing the mean).
- IQR = Q3 - Q1; outlier limits Q1 - 1.5 IQR and Q3 + 1.5 IQR.
- Standardise: z = (x - mean) / std.
- Neuron: output = activation(sum of weight x input + bias).

How to answer any viva question (a simple 3-step pattern)

1. **Define** it in one sentence.
2. **Give an example** from your own practical ("in my spam code, ...").
3. **Say why it matters** ("... otherwise the model would ...").

If you do not know an answer: do not stay silent. Say what you do know about the topic, then "I am not fully sure about the rest, sir." Examiners reward honesty and partial understanding far more than silence.

Exam-room checklist (Ubuntu)

- Open a terminal with Ctrl + Alt + T, make a folder, copy the CSV into it.
- Test imports first: `python3 -c "import pandas, seaborn, sklearn; print('OK')"`.
- Type or paste the code into a file, **save it** (a file that is not saved is empty, exactly like your `sales.py` earlier), then run `python3 file.py`.
- Close each plot window to continue.
- Run `ls` to confirm the CSV and the `.py` file are both in the folder.
- If it breaks, read the **last line** of the error; it names the problem.

Be ready to explain your output. For every practical, know the three things the examiner will point at: the confusion matrix or score printed, which model did better, and why.

Good luck for tomorrow. You have the code, the concepts and the answers.